5장. 회원가입 구현하기

본 장에서는 사용자 인증의 출발점인 회원가입 기능을 구현합니다. 대형 프로젝트 개발을 위한 **APIRouter 기반 라우터 구조화** 기법을 학습하고, 사용자의 비밀번호를 해시 암호화(bcrypt)하여 데이터베이스에 안전하게 기록하는 설계를 알아봅니다.

5.1 비밀번호 단방향 해싱 아키텍처

비밀번호는 데이터베이스 유출 사태를 대비해 절대로 평문 상태로 데이터베이스에 보관해서는 안 됩니다. **단방향 해시 함수(One-Way Hash Function)**는 원본 값을 입력값으로 주었을 때 암호화된 메시지 다이제스트를 도출하지만, 그 암호문을 복호화하여 비밀번호를 다시 원상 복구하는 것은 불가능한 구조입니다. 본 실습에서는 대표적인 안전 해싱 규격인 **bcrypt**를 사용합니다.

5.2 User 데이터 모델링 및 테이블 정의

회원을 관리할 테이블 구조를 정의합니다. 고유한 식별자인 id 와 중복 가입을 방지할 고유 인덱스가 부여된 email , 그리고 해시 처리될 hashed_password 로 설계합니다.

```
# models.py (User 추가 버전)
from sqlalchemy import Column, Integer, String, Boolean
from database import Base

class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(100), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    is_active = Column(Boolean, default=True)
```

5.3 APIRouter를 사용한 라우터 구조화

모든 API를 `main.py` 한 곳에 기재하면 유지보수가 매우 어려워집니다. 기능별로 라우터를 분리해 가독성을 높입니다.

```
# routers/user.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from database import SessionLocal
from pydantic import BaseModel, EmailStr

router = APIRouter(
    prefix="/users",
    tags=["users"] # Swagger UI의 카테고리 태그 분류
)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# 🔑 회원가입 데이터 요청/응답 스키마
class UserCreate(BaseModel):
    email: EmailStr # EmailStr 형식을 사용하려면 pydantic[email] 설치 필요
    password: str

class UserResponse(BaseModel):
    id: int
    email: EmailStr
    is_active: bool

    class Config:
        from_attributes = True
```

5.4 비밀번호 해싱 및 회원가입 API 로직 구현

비밀번호 해시 라이브러리인 `passlib`과 `bcrypt`를 활용하여 해싱 함수를 정의하고, 이메일 중복 체크를 거쳐 데이터베이스에 신규 유저를 등록하는 API를 완성합니다.

```python
# security.py
from passlib.context import CryptContext

# bcrypt 알고리즘을 사용한 해시 컨텍스트 초기화
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def get_password_hash(password: str) -> str:
    """평문 비밀번호를 단방향 암호 해싱하여 반환합니다."""
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """입력 비밀번호와 DB의 해시 비밀번호가 일치하는지 비교 검증합니다."""
    return pwd_context.verify(plain_password, hashed_password)
```

```python
# routers/user.py (회원가입 API 추가)
from security import get_password_hash
from models import UserModel

@router.post("/signup", response_model=UserResponse, status_code=201)
def signup(user: UserCreate, db: Session = Depends(get_db)) -> UserModel:
    # 1. 중복된 이메일이 이미 존재하는지 체크
    existing_user = db.query(UserModel).filter(UserModel.email == user.email).first()
    if existing_user:
        raise HTTPException(
            status_code=400,
            detail="이미 사용 중인 이메일 주소입니다."
        )

    # 2. 평문 패스워드 해싱
    hashed_pw = get_password_hash(user.password)

    # 3. 데이터 저장
    new_user = UserModel(
        email=user.email,
        hashed_password=hashed_pw
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user
```

5.5 메인 애플리케이션 연결

작성한 라우터 객체를 `main.py`에 포함시켜 라우팅을 등록합니다.

```
# main.py
from fastapi import FastAPI
from database import engine, Base
import routers.user as user_router

# 테이블 동적 생성
Base.metadata.create_all(bind=engine)

app = FastAPI(title="FastAPI 백엔드 서비스")

# 라우터 연결
app.include_router(user_router.router)
```